

Practica 3 ---> MOISÉS MONTES IGLESIAS

---> UO296102

Ejercicio 1:

Tras analizar la complejidad de las tres clases anteriores, no se le pide hacer sus tablas de tiempos, pero sí razonar si los tiempos coinciden o no la complejidad temporal de cada algoritmo.

Contestar: ¿para qué valor de n dejan de dar tiempo (abortan) las clases `Sustraccion1` y `Sustraccion2`? ¿por qué sucede eso?

SOLUCIÓN:

La clase `Sustraccion1`, tiene una complejidad $O(n)$, ya que siguiendo la regla de Divide y Vencerás por sustracción, $a = 1$, $b = 1$, $k = 0$. Por tanto, como $a = 1$, equivale a $O(n^{(k+1)})$, que sustituyendo es $O(n)$. Tras aplicar la medición de los tiempos, los resultados obtenidos siguen una complejidad lineal.

La clase `Sustraccion2`, tiene una complejidad $O(n^3)$, ya que siguiendo la regla de Divide y Vencerás por división, $a = 1$, $b = 1$, $k = 2$. Por tanto, como $a = 1$, equivale a $O(n^{(k+1)})$, que sustituyendo es $O(n^3)$.

La clase `Sustraccion3`, tiene una complejidad $O(2^n)$, ya que siguiendo la regla de Divide y Vencerás por sustracción, $a = 2$, $b = 1$, $k = 0$. Por tanto, como $a = 1$, equivale a $O(a^{(n/b)})$ que sustituyendo es $O(2^n)$.

Para los valores de n mayores de 8000 tanto para `Sustraccion1` como para `Sustraccion2`. Y esto se debe a que al utilizar el tipo `long` para el valor de n , este no permite números mayores que $2^{31} - 1$ bits.

Calcular: ¿cuántos años tardaría en finalizar la ejecución `Sustraccion3` para $n=80$?

SOLUCIÓN:

```
n=20**TIEMPO=77**nVeces=1
n=21**TIEMPO=151**nVeces=1
n=22**TIEMPO=302**nVeces=1
n=23**TIEMPO=623**nVeces=1
n=24**TIEMPO=1221**nVeces=1
n=25**TIEMPO=2461**nVeces=1
```

$$T2 = (O(n2) * t1) / O(n1) = (2^{(n2)} * 2461) / (2^{(n1)}) = (2^{(80)} * 2461) / (2^{(25)}) = 2461 * 2^{55}$$

Este valor es en milisegundos, si lo pasamos a años, obtenemos un valor de 2808 años.

Debemos tener en cuenta que el valor del parámetro que hemos dado para este ejemplo es 1.

Implementar una clase `Sustraccion4.java` con una complejidad $O(n^3)$ (con su pertinente toma de tiempos) y después rellenar una tabla en la que se muestre el tiempo (en msg.) para $n=100$, 200, 400, 800, ... (hasta `FdT`)

SOLUCIÓN:

n	Sustraccion4
100	4 ms = 0,004 s --> FdF < 50 ms
200	35 ms = 0,035 s --> FdF < 50 ms
400	283 ms = 0,283 s
800	2244 ms = 2,244 s
1600	18398 ms = 18,398 s
3200	FdT > 60 ms



Implementar una clase Sustraccion5.java con una complejidad $O(3n/2)$ (con su pertinente toma de tiempos) y después rellenar una tabla en la que se muestre el tiempo (en msg.) para $n=30, 32, 34, 36, \dots$ (hasta FdT)

SOLUCIÓN:

n	Sustraccion5 (ms)
30	734 ms = 0,734 s
32	2165 ms = 2,165 s
34	6685 ms = 6,685 s
36	19852 ms = 19,852 s
38	59534 ms = 59,534 s
40	FdT > 60 s



Calcular: ¿cuántos años tardaría en finalizar la ejecución Sustraccion5 para n=80?

SOLUCIÓN:

$$T2 = (O(n2) * t1) / O(n1) = (3^{(n2/2)} * 734) / (3^{(n1/2)}) = (3^{(80/2)} * 734) / (3^{(30/2)}) = 734 * 3^{25}.$$

Este valor es en milisegundos, si lo pasamos a años, obtenemos un valor de 19734 años.

Debemos tener en cuenta que el valor del parámetro que hemos dado para este ejemplo es 1.

Ejercicio 2:

Tras analizar la complejidad de las tres clases anteriores, no se le pide hacer sus tablas de tiempos, pero sí razonar si los tiempos coinciden o no la complejidad temporal de cada algoritmo. Implementar una clase Division4.java con una $O(n^2)$ (con $a < b^k$) y la pertinente

toma de tiempos. Después rellenar una tabla en la que se muestre el tiempo (en msg.) para $n=1000, 2000, 4000, 8000, \dots$ (hasta FdT).

SOLUCIÓN:

La clase Division1, tiene una complejidad $O(n)$, ya que siguiendo la regla de Divide y Vencerás por división, $a = 1, b = 3, k = 1$. Por tanto, $a < b^k$ que equivale a $O(n^k)$, que sustituyendo es $O(n)$. Tras aplicar la medición de los tiempos, los resultados obtenidos siguen una complejidad lineal.

La clase Division2, tiene una complejidad $O(n * \log(n))$, ya que siguiendo la regla de Divide y Vencerás por división, $a = 2, b = 2, k = 1$. Por tanto, $a = b^k$ que equivale a $O(n^k * \log(n))$, que sustituyendo es $O(n * \log(n))$.

La clase Division3, tiene una complejidad $O(n^{\log_2(2)})$, ya que siguiendo la regla de Divide y Vencerás por división, $a = 2, b = 2, k = 0$. Por tanto, $a > b^k$ que equivale a $O(n^{\log_b(a)})$, que sustituyendo es $O(n^{\log_2(2)})$.

n	Division4
1000	14 ms = 0,014 s
2000	61 ms = 0,061 s
4000	245 ms = 0,245 s
8000	996 ms = 0,996 s
16000	3984 ms = 3,984 s
32000	16061 ms = 16,061 s
64000	FdT > 60



Implementar una clase Division5.java con una complejidad $O(n^2)$ (con $a > bk$) y la pertinente toma de tiempos. Después rellenar una tabla en la que se muestre el tiempo (en msg.) para $n=1000, 2000, 4000, 8000, \dots$ (hasta FdT).

SOLUCIÓN:

n	Division5
1000	52 ms = 0,052 s
2000	197 ms = 0,197 s
4000	829 ms = 0,829 s
8000	3271 ms = 3,271 s
16000	12882 ms = 12,882 s
32000	52130 ms = 52,13 s
64000	FdT > 60



Ejercicio 3:

Tras analizar la complejidad de los diversos algoritmos que hay dentro de las dos clases, ejecutarlas y tras poner en una tabla los tiempos obtenidos, comparar la eficiencia de cada algoritmo.

SOLUCIÓN:

El algoritmo VectorSuma, consta de 3 formas de realizar las sumas de sus valores.

En todas ellas, tenemos una complejidad $O(n)$, pero realizando las sumas de diversas formas.

El algoritmo de Fibonacci está realizado de 4 formas, pero todas ellas tienen una complejidad $O(n)$, salvo la última, la cual consta de una complejidad $O(1,6^n)$, en el que notamos una diferencia muy grande de complejidad respecto del resto, pero lo tendremos en cuenta igual dentro de los límites para nuestras mediciones.

n	Fibonacci(metodo1)	Fibonacci(metodo2)	Fibonacci(metodo3)	Fibonacci(metodo4)
40	361 ns	591 ns	729 ns	6555 ms
n	VectorSuma(metodo1)	VectorSuma(metodo2)	VectorSuma(metodo3)	
100	710 ns	1915 ns	4915 ns	

Comenzando por Fibonacci, vemos que el método con complejidad $O(1,6^n)$, nos da un valor muchísimo mas grande que el resto, solamente con fijarnos en la escala de medición, (ns = 10^{-9} , ms = 10^{-3}). El más eficiente es el metodo1, ya que, trabajando con el mismo tamaño de problema, obtenemos una medición de tiempo más rápida que el resto.

Para el caso del VectorSuma, estamos en una situación similar ya que pese a que todas tengan una complejidad $O(n)$, el primer método realiza la operación más rápido que el resto, pese a ser el mismo tamaño.

	Fib(metodo2)/Fib(metodo1)	Fib(metodo3)/Fib(metodo2)	Fib(metodo3)/Fib(metodo1)
Eficiencia	1,6371	1,2335	2,0194
	VS(metodo2)/VS(metodo1)	VS(metodo3)/VS(metodo2)	VS(metodo3)/VS(metodo1)
Eficiencia	2,6972	2,5666	6,9225

Aquí muestro la diferencia de tiempos de un método a otro. Es decir, por ejemplo en Fib(metodo2)/Fib(metodo1), el método 2 es un 1,6371 veces más lento que el método 1.

Ejercicio 4:

Implementar una clase MezclaTiempos.java que tras ser ejecutada sirva para rellenar la siguiente tabla (análoga a las vistas en la práctica 2):

SOLUCIÓN:

n	t ordenado	t inverso	t aleatorio
31250	19 ms = 0,019 s --> FdF < 50 ms	19 ms = 0,019 s --> FdF < 50 ms	22 ms = 0,022 s --> FdF < 50 ms
62500	35 ms = 0,035 s --> FdF < 50 ms	35 ms = 0,035 s --> FdF < 50 ms	44 ms = 0,044 s --> FdF < 50 ms
125000	74 ms = 0,074 s	75 ms = 0,075 s	88 ms = 0,088 s
250000	154 ms = 0,154 s	158 ms = 0,158 s	194 ms = 0,194 s
500000	322 ms = 0,322 s	335 ms = 0,335 s	395 ms = 0,395 s
1000000	668 ms = 0,668 s	701 ms = 0,701 s	828 ms = 0,828 s
2000000	1399 ms = 1,399 s	1466 ms = 1,466 s	1691 ms = 1,691 s
4000000	3023 ms = 3,023 s	2989 ms = 2,989 s	3489 ms = 3,489 s
8000000	5964 ms = 5,964 s	6168 ms = 6,168 s	7282 ms = 7,282 s
16000000	12217 ms = 12,217 s	12699 ms = 12,699 s	14843 ms = 14,843 s
32000000	25234 ms = 25,234 s	26506 ms = 26,506 s	30547 ms = 30,547 s
64000000	51928 ms = 51,928 s	54658 ms = 54,658 s	FdT > 60
128000000	FdT > 60	FdT > 60	FdT > 60



Rellene la tabla siguiente, trasladando a ella los tiempos obtenidos para el Rápido en el caso aleatorio en la práctica 2 y los obtenidos aquí para el Mezcla en el caso aleatorio. ¿Qué constante le sale como comparación de ambos algoritmos?

SOLUCIÓN:

n	t mezcla(t1)	t rapido(t2)	t1/t2
250000	194 ms = 0,194 s	184 ms = 0,184 s	1,0543
500000	395 ms = 0,395 s	373 ms = 0,373 s	1,0590
1000000	828 ms = 0,828 s	759 ms = 0,759 s	1,0909
2000000	1691 ms = 1,691 s	1652 ms = 1,652 s	1,0236
4000000	3489 ms = 3,489 s	3550 ms = 3,55 s	0,9828
8000000	7282 ms = 7,282 s	7931 ms = 7,931 s	0,9182
16000000	14843 ms = 14,843 s	14884 ms = 14,884 s	0,9972



Para hallar la constante obtenida al comparar ambos logaritmos, cogemos los datos obtenidos de t_1 / t_2 , y los dividimos por el numero par de valores que hemos medido (es decir 7).

Constante $t_1/t_2 = (1,0543 + 1,059 + 1,0909 + 1,0236 + 0,9828 + 0,9182 + 0,9972) / 7 = 1,018$.

Por lo tanto, la constante de comparación promedio de los algoritmos es 1,018; es decir, el algoritmo mezcla y rápido tardan prácticamente lo mismo y constan de la misma complejidad.